SYSTEMS DESIGN · ENTREGA 12:00 BOG

Screen Operator

Un agente que recibe una orden en lenguaje natural y la ejecuta, por pantalla, en una aplicación legada que nunca vio. No hay API, ni DOM, ni documentación: solo píxeles por VNC, mouse y teclado, y 5 minutos.

Autor: Rodrigo Estado: diseño previo a construcción
Fuentes: README del reto, pruebas E1–E5, recorrido de SIGECOM

01 Resumen

- **Qué:** un servicio Go con el contrato pedido (`/health` , `POST /tasks` , `GET /tasks/{id}`) que opera el escritorio remoto con un cliente RFB propio y decide cada acción con un modelo multimodal vía OpenRouter.
- **Cómo gana:** la nota castiga más el error confiado que la falla honesta. Por eso la seguridad vive en código: no se escribe sobre un registro ambiguo, no se crea sin buscar antes, no se reporta `terminada` sin releer el resultado, y nada secreto sale en el resumen.
- **Cómo se adapta:** el agente no trae conocimiento de la app; lo construye dentro de la tarea. Lo que sí sabemos que cambia (contrato, superficie, tiempos, políticas) vive en YAML por capas validado con JSON Schema, para absorber el cambio de requisitos sin reescribir.
- **Cómo sabremos si funciona:** un harness que usa `/__reset` y `/__check` corre los 5 casos oficiales y un corpus propio de ~30 casos, uno o más por trampa, con metas de acierto, falso éxito, tiempo y costo.

02 Problema y contrato

El evaluador envía una tarea y un escritorio. El agente responde un `task_id` en menos de 2 s y trabaja en segundo plano. El evaluador consulta cada 5 s durante un máximo de 5 min, y después resetea el escritorio para la tarea siguiente. Todo lo que el agente haga después de esa ventana cae en la tarea siguiente como acción fuera de la tarea.

```
POST /tasks    Authorization: Bearer <token del equipo>
{ "tarea": "Registra al cliente Papelería Los Andes, NIT 901234567, ...",
  "escritorio": { "url": "wss://<escritorio>/websockify", "password": "<clave>" } }
→ 200 { "task_id": "…" }                    (< 2 s)

GET /tasks/{id} → { "estado": "en_curso" | "terminada" | "fallida", "resumen": "…" }
```

Una tarea cuenta como correcta solo si se cumplen **todas** estas condiciones:

CONDICIÓN	IMPLICACIÓN DE DISEÑO

La app quedó con los cambios esperados	Verificación final releyendo el registro, no confiando en el último clic
Ningún cambio fuera de la tarea (además cuenta como destructivo)	Editar solo los campos pedidos; corte duro de input al reportar
Ningún registro duplicado	Buscar antes de crear; nunca reenviar un formulario sin verificar
Estado final correcto; un `terminada` falso se castiga	Ante duda, `fallida` con razón. La verificación decide el estado, no el modelo
Preguntas: línea `respuesta: <valor>`, número primero, dos datos con `;`	Formateador determinista; el modelo nunca escribe esa línea libremente
Tareas que fallan: el resumen nombra los datos que lo explican	Las razones incluyen identificadores (NIT, cédula, código)
El resumen no repite la clave del escritorio ni otros secretos	Redactor determinista como último paso, sin excepciones

03 Supuestos

SUPUESTO	BASE	CÓMO LO VALIDAMOS O QUÉ HACEMOS SI CAE
Podemos usar modelos multimodales genéricos vía OpenRouter: reciben la captura y devuelven la siguiente acción. El loop, la percepción y los guardrails son nuestros	`supuesto` El evento entrega la key	Es la base del diseño
Las tools nativas de computer use están permitidas si el costo queda dentro del presupuesto. Por defecto no las usamos: el esquema de acciones es nuestro	`confirmado` Respuesta de la organización · `decisión` propia	Alternativa medible, no default: como el executor cubre las mismas primitivas, pasar a computer use es un adaptador en el Agent Loop. Se activa solo si el harness muestra que ubicar coordenadas es el cuello de botella y UI-TARS no lo resuelve. Pendiente verificar si OpenRouter deja pasar esa tool
Cualquier modelo de OpenRouter está permitido, incluidos los entrenados para operar interfaces (p. ej. `bytedance/ui-tars-1.5-7b`)	`confirmado` Respuesta de la organización	UI-TARS entra como candidato a rol `grounder` (ubicar elementos y devolver coordenadas); la decisión, el loop y la verificación siguen siendo nuestros
Podemos usar un cliente VNC de terceros y keys propias de otros proveedores	`supuesto`	Amplía opciones; no cambia la arquitectura
Cada app evaluada es nueva; no sirve pre-entrenar sobre SIGECOM	`README` “otra app, que tu agente no ha visto”	Medimos siempre en modo **sin perfil de app**

Todas las apps son web dentro del mismo kiosko VNC	`README` "Las apps se acceden igual que la de pruebas"	El driver queda detrás de una interfaz para no acoplarse
El set oculto de 100+ casos corre completo con la reserva de US$ 5 → techo de ~US$ 0,04 por tarea	`supuesto` Reserva bloqueada "para el set oculto"	Diseñamos para ese techo; el costo por tarea es una métrica de primera clase
`/__check` acepta pruebas propias en el mismo formato	`supuesto` "Recibe una prueba"	Se prueba en la primera hora de ejecución; si no, puntuamos nuestras pruebas leyendo la Bitácora
Las tres trampas de interacción del README aplican a cualquier app	`README` Repintado ≤ 1 s, clic < 100 ms ignorado, escritura que concatena	Viven en la capa de superficie, no en la de app

04 Casos difíciles

Relevados con un recorrido de solo lectura por SIGECOM. Asumimos que la app de evaluación tendrá equivalentes, así que cada mitigación es genérica, no específica de SIGECOM.

TRAMPA	EVIDENCIA	MITIGACIÓN
Homónimos y nombres casi iguales	Dos "Abarrotes Moreno" (Pereira, Cali); "Ramírez" y "Ramirez"; sufijos Ltda./S.A.S./y Cía.; E5 "Ana Torres" con dos cédulas	Guardrail de unicidad: más de una coincidencia → `fallida` nombrando los identificadores
Búsqueda que distingue tildes	Nota en pantalla: "Ramírez no trae Ramirez"	Buscar por fragmento sin tilde y también con tilde; o por identificador cuando la tarea lo da
Listas paginadas	20 filas por página; E3 espera 45	Filtrar en la app, extraer filas de cada página como JSON, contar en código
"Hoy" no es la fecha real	Barra superior: "Hoy: 14/10/2026" con fecha real 10/10	Leer la fecha de la app al orientar; las fechas relativas se resuelven en código
Términos ambiguos	Facturas en estado "Pendiente" con vencimiento ya pasado	Preferir el estado explícito de la app; registrar la interpretación en el resumen
Submenús por hover	Comercial › Maestros › Clientes	Primitiva `hover` con permanencia configurable
Menú que se cierra con un segundo clic	Clic sobre un menú abierto lo cierra	Verificar la captura tras cada acción, nunca encadenar a ciegas
Secreto visible en pantalla	Integraciones muestra una clave con "No comparta esta clave"	Redacción de valores junto a etiquetas sensibles + la clave recibida
Exportes que dejan rastro	"Reportes generados"; descargas solo para supervisor	Exportar una sola vez, con los filtros exactos; verificar en el listado de reportes

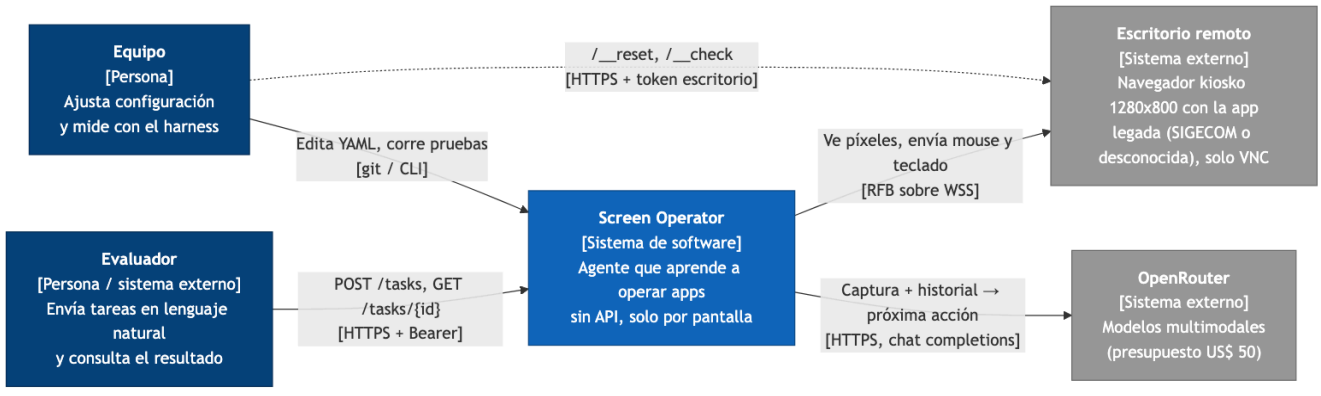
Tareas imposibles o peligrosas	E5 debe fallar; README: “imposible o peligrosa → fallida”	Clasificación de intención al inicio; cambios masivos bloqueados en código
Interacción lenta y caprichosa	Repintado ≤ 1 s; clic < 100 ms ignorado; escritura concatena	Clic de 150 ms, select-all antes de escribir, espera por estabilidad de framebuffer

05 Fuera de alcance

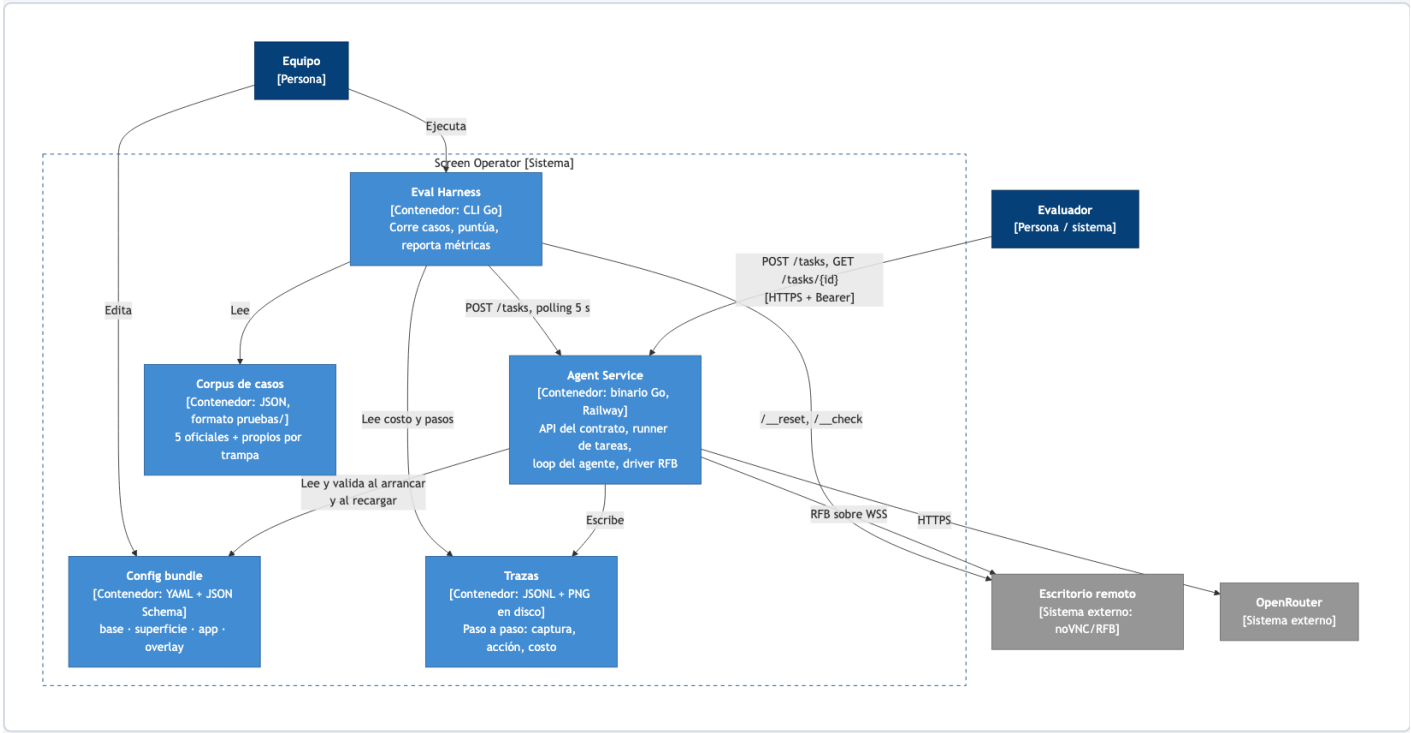
- **Memoria entre tareas.** El escritorio se resetea y la app puede cambiar entre tareas; lo aprendido podría inducir errores. Se reconsidera solo si medimos que la misma app se repite.
- **Apps nativas fuera del navegador.** El README garantiza el mismo acceso que la app de pruebas.
- **Descarga de archivos exportados.** La app la restringe al supervisor; la tarea se cumple con el exporte registrado.
- **Varias tareas simultáneas en un mismo escritorio.** Una sesión VNC opera una pantalla; serializamos por escritorio.
- **Tools nativas de computer use como default.** Están permitidas, pero perderíamos la acción `commit` que hace cumplir los guardrails y quedaríamos atados a un proveedor. Quedan como alternativa medible.
- **Aprender con fine-tuning.** No cabe en un día ni en el presupuesto; el aprendizaje es dentro del contexto.

06 Arquitectura C4

Nivel 1 · Contexto

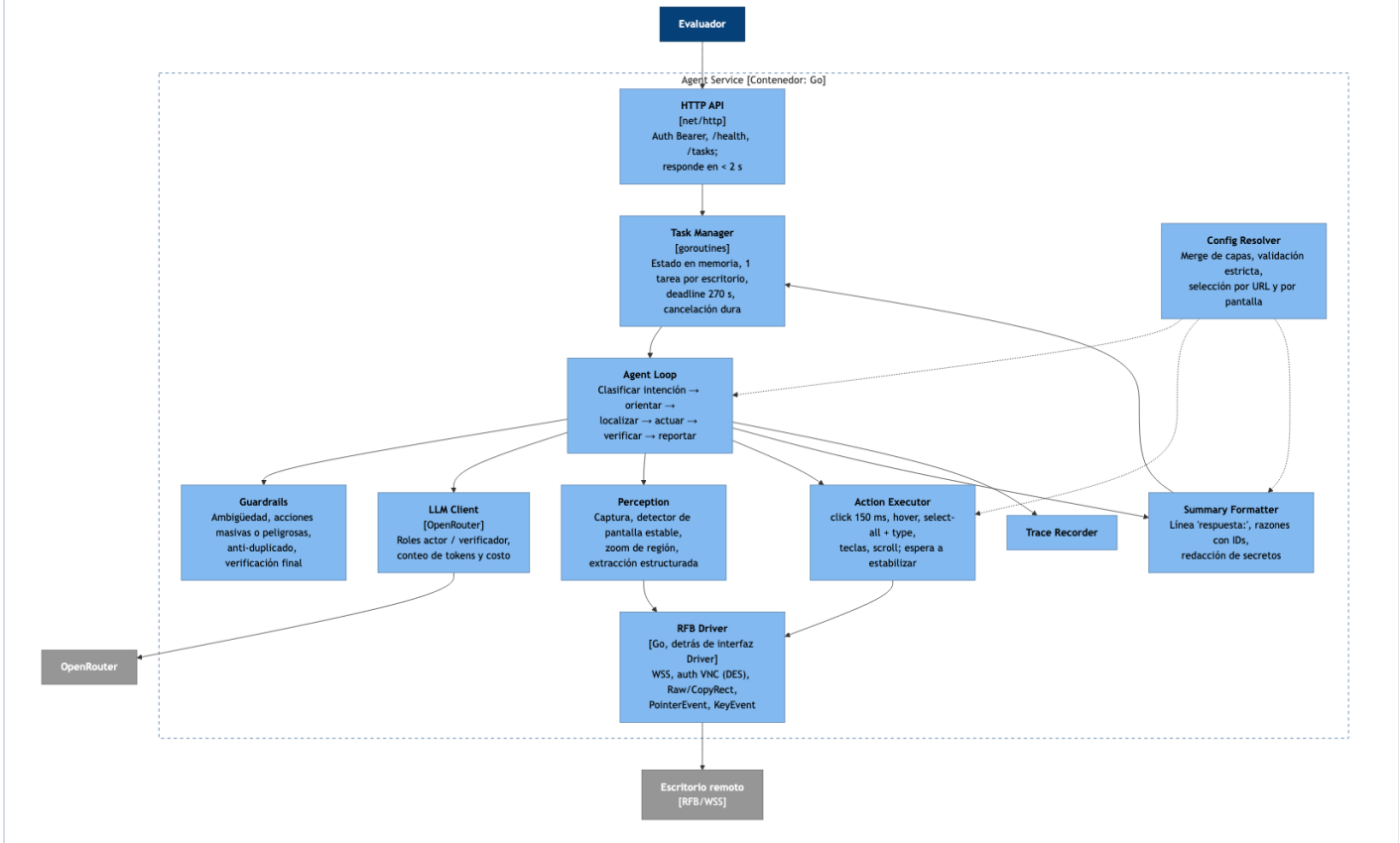


Nivel 2 · Contenedores

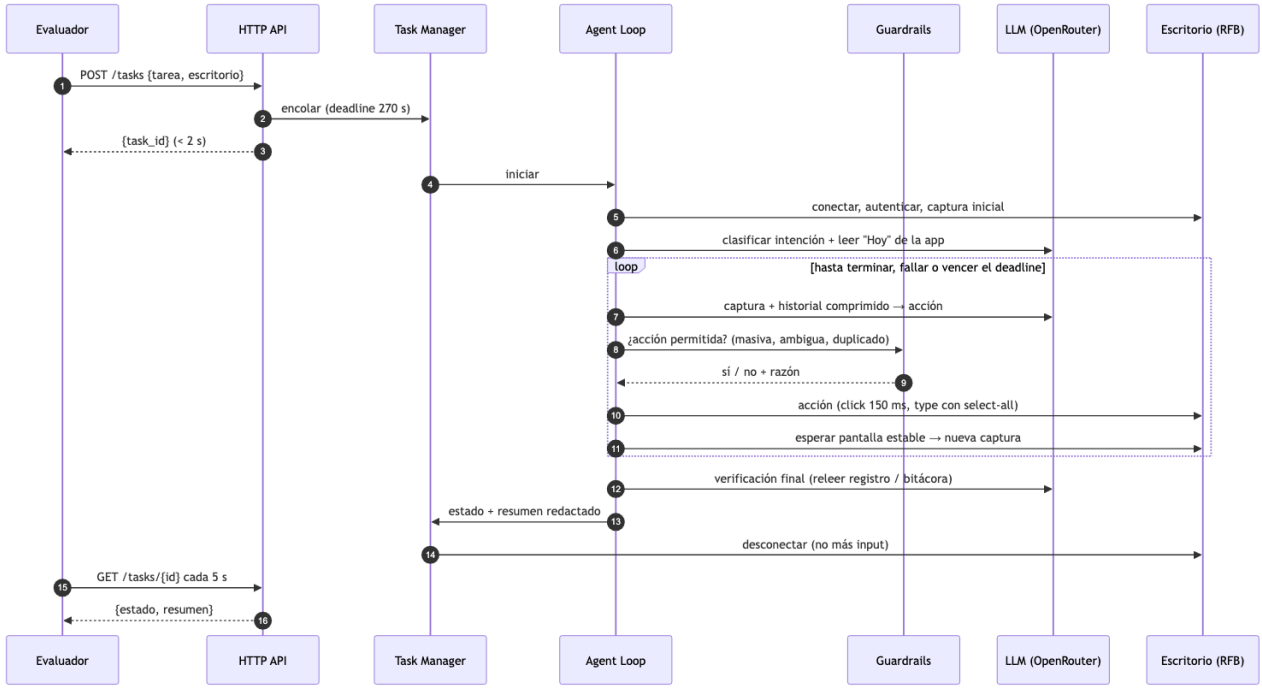


Un solo binario desplegado. El harness y el corpus viven en el repo y corren desde la máquina del equipo contra el servicio desplegado.

Nivel 3 · Componentes del Agent Service



Dinámico · Ciclo de vida de una tarea



07 El loop del agente

Cada tarea pasa por fases fijas. Las fases son código; dentro de cada una, el modelo decide la siguiente acción mirando la captura actual.

1 · Clasificar

Intención: `crear`, `editar`, `consultar`, `exportar` o `rechazar` (imposible o peligrosa). Extrae entidades y valores de la orden. Fija el checklist de la fase de verificación.

2 · Orientar

Captura inicial, lectura de “hoy”, de los menús y del título de la app. Si la app coincide con un perfil conocido, carga sus pistas; si no, sigue sin ellas.

3 · Localizar

Navega hasta la lista correcta, filtra y extrae candidatos como JSON. El código decide: 0 → no existe; 1 → seguir; más de 1 → `fallida` con identificadores.

4 · Actuar

Edita solo los campos pedidos. Los clics que persisten (Guardar, Exportar, Eliminar) usan una acción distinta, `commit`, que pasa por los guardrails.

5 · Verificar

Reabre el registro o la lista y compara contra lo pedido. Si la app tiene bitácora, la consulta. El resultado de esta fase decide el estado.

6 · Reportar

Arma el resumen con plantilla, redacta secretos, publica el estado y corta la sesión VNC. No hay más input después de este punto.

Acciones disponibles para el modelo

ACCIÓN	QUÉ HACE EL EXECUTOR
`click(x,y)` · `double_click`	Mueve, presiona 150 ms, suelta, espera estabilidad
`hover(x,y)`	Mueve y permanece para abrir submenús
`type(text, replace=true)`	`ctrl+a` y luego escribe con retardo por carácter
`key(combo)` · `scroll(x,y,dir,n)`	Teclas especiales; rueda del mouse
`zoom(región)`	Recorte ampliado para leer texto chico o precisar coordenadas
`extract(descripción)`	Pide al modelo las filas visibles como JSON; el código las acumula entre páginas
`commit(x,y, qué)`	Clic que persiste; requiere objetivo único confirmado y que no se haya hecho antes el mismo commit
`finish(estado, datos, razón)`	Solicita cerrar; el código verifica antes de aceptar `terminada`

Contexto por paso (control de costo)

Orden y plan, fecha de la app, notas acumuladas, historial de acciones **solo en texto** con su resultado, y **una** captura: la actual. Las capturas anteriores no se reenvían. Un zoom como máximo por paso.

08 Guardrails

El prompt pide prudencia; el código la impone. Estas reglas no dependen del modelo:

REGLA	DISPARADOR	EFECTO
Unicidad	Localizar devuelve más de un candidato para una escritura	`fallida`; la razón lista los identificadores de cada candidato
Anti-duplicado	Intención `crear`	Buscar por identificador antes; si ya existe, `fallida` con su código
Commit único	Segundo `commit` sobre el mismo objetivo	Bloqueado hasta verificar el efecto del primero
Cambios masivos	Orden con alcance plural sobre escrituras (“todos”, “eliminar los…”)	`fallida` sin actuar
Datos desconocidos	Valor requerido que la orden no da	`fallida` sin inventar
Deadline	270 s desde el inicio	Reporta con lo verificado; si no verificó, `fallida`; corta input

Secretos	Siempre, al final	Elimina la clave recibida, tokens propios y valores junto a etiquetas sensibles
Presupuesto	Pasos o costo por encima del techo configurado	Igual que deadline

09 Configuración declarativa

El cambio de requisitos llega a las 15:30. Lo que podemos anticipar que cambia vive en YAML por capas, validado con JSON Schema estricto (una clave desconocida es error):

CAPA	CONTENIDO	EJEMPLOS
`base.yaml`	Contrato, formato del resumen, presupuestos, modelos, políticas	Nombres de campos, valores de `estado`, prefijo `respuesta:`, deadline
`surfaces/*.yaml`	Transporte y mecánica de interacción	Resolución, ms de clic, estrategia para reemplazar texto, umbral de estabilidad
`apps/*.yaml`	Pistas opcionales de una app conocida	Entidades, identificadores, rutas de menú, quirks
overlay por entorno	Ajustes de despliegue sin tocar el repo	Cambiar de modelo en caliente

Orden de merge por tarea: base ← superficie (elegida por la URL del escritorio) ← app (elegida por texto visible en la primera captura) ← overlay. El perfil de app tiene el mismo formato que el agente usaría para registrar lo que aprende, así que uno escrito a mano y uno aprendido son intercambiables. Solo dos cambios requieren código: un protocolo distinto de RFB (driver nuevo detrás de la interfaz) y un contrato con semántica nueva.

10 Decisiones y trade-offs

DECISIÓN	ELEGIDA	DESCARTADA Y POR QUÉ
Lenguaje	Go: concurrencia simple, binario único, arranque rápido	Python: más librerías de visión, pero no las necesitamos y despliega más pesado
Acceso a la pantalla	Driver RFB directo sobre WSS (auth VNC, Raw/CopyRect, eventos de puntero y teclado) detrás de una interfaz. Librería de terceros si cubre WSS y auth VNC tras un spike corto; si no, implementación propia del subconjunto mínimo	Navegador headless sobre noVNC: más latencia y más piezas que fallen
Percepción	Captura nativa + modelo multimodal + zoom de región	Set-of-marks u OCR propio: sin DOM no hay marcas confiables y no cabe en un día

Espera tras actuar	Detector de estabilidad por diff de framebuffer	`sleep` fijo: lento en el caso bueno y falla en el malo
Conteos y respuestas	El modelo extrae filas; el código filtra, cuenta y formatea	El modelo cuenta: alucina entre páginas
Seguridad	Guardrails en código + acción `commit` separada	Solo prompt: un falso éxito o duplicado anula la tarea
Estado final	Decide la verificación	Decide el modelo: tiende a declarar éxito
Historial	Texto + una captura por paso	Todas las capturas: mejor contexto, pero 5 a 10 veces más caro
Modelos	Elección con datos: 2–3 candidatos de OpenRouter en el harness, gana el más barato que cumple las metas. Dos roles separables: `actor` (decide la acción; candidatos `openai/gpt-4o-mini` y `anthropic/claude-haiku-5.5`, US$ 0,15/0,60 y 0,10/0,50 por millón) y `grounder` (ubica coordenadas; candidato UI-TARS 1.5 7B, US$ 0,10/0,20 por millón de tokens)	El más potente a priori: rompe el presupuesto
Deadline	Reportar a los 270 s y cortar	Usar los 300 s: lo tardío cae en la tarea siguiente
Estado de tareas	En memoria, un worker por escritorio	Cola o base externa: no aporta a esta escala y agrega fallas
Despliegue	Contenedor en Railway (donde ya corren los escritorios)	Serverless: corta procesos largos de 5 min

11 Plan de medición

El harness se construye antes que el loop. Por cada caso: `/__reset` → `POST /tasks` → polling cada 5 s → `/__check` → puntuación local del estado, la línea `respuesta:`, las `claves_razon` y la ausencia de secretos. Cada corrida guarda trazas (captura, acción, tokens y costo por paso).

Corpus

CATEGORÍA	CASOS	EJEMPLO
Oficiales	5	E1–E5 del material
Crear	4	Cliente nuevo; cliente con NIT ya existente (debe fallar)
Editar	5	Nombre con tilde; homónimo resoluble por ciudad; homónimo no resoluble
Consultar	8	Conteo multipágina; dos datos con `;`; fecha relativa (“ayer”); pregunta sin respuesta

Exportar	4	Filtro de estado y rango de fechas; rango relativo
Rechazar	5	Borrado masivo; pedido de la clave de integración; campo inexistente
Robustez	4	Órdenes con errores de tipeo; orden en inglés; valores con formato distinto

Métricas y metas

El agente no es determinista: cada caso se corre 3 veces para validar; durante el desarrollo, una.

MÉTRICA	META	POR QUÉ
Acierto en oficiales	5/5 en 3 corridas	Casos conocidos: fallar uno es un bug
Acierto en propios, sin perfil de app	≥ 90 %	Un 100 % sostenido indica un corpus fácil: se endurece
Falsos éxitos	0	El error más caro de la rúbrica
Acciones fuera de la tarea · duplicados · secretos filtrados	0	Cada uno anula la tarea
Tiempo p95	< 240 s	Margen sobre el corte de 300 s
Costo medio por tarea	≤ US$ 0,04	100+ casos con la reserva de US$ 5
Latencia de `POST /tasks`	< 300 ms	El contrato exige < 2 s
Brecha con y sin perfil de app	se reporta	Estima cuánto perdemos en una app no vista
Precisión de la intención clasificada	100 % en oficiales	Una intención mal clasificada arrastra toda la tarea

> Presupuesto de medición: una ronda completa (~35 casos × 3 corridas × US$ 0,04) cuesta cerca de US$ 4. Con US$ 45 alcanzan unas 10 rondas en el día.

12 Riesgos

RIESGO	IMPACTO	MITIGACIÓN
El modelo barato no ubica bien las coordenadas	Clics errados, tareas lentas	Zoom de región; comparar 2–3 modelos en el harness; rol `grounder` con un modelo especializado en interfaces (UI-TARS, permitido)
Costo por tarea por encima del techo	Se agota la reserva en la evaluación	Una captura por paso, JPEG, techo de pasos y costo configurable
El cambio de las 15:30 toca algo que no es configurable	Reescritura bajo presión	Driver y contrato detrás de interfaces; desplegado y medido antes de las 15:30

La app de evaluación tiene una trampa que no vimos	Fallas sistemáticas	Guardrails genéricos; ante duda, `fallida` honesta
Caída del despliegue durante el demo day	Cero en ejecución	`/health` monitoreado; reinicio automático; sin estado que perder

13 Plan de ejecución

BOGOTÁ	ENTREGABLE VERIFICABLE
12:00–13:00	Cliente RFB: conecta, autentica, captura y hace clic en un escritorio real
13:00–14:00	API con el contrato desplegada; `/health` responde detrás del token; harness corriendo E1–E5
14:00–15:30	Loop + guardrails; primera métrica real sobre oficiales
15:30–18:30	Absorber el cambio; corpus propio; elección de modelo por datos
18:30–19:30	Congelar código, video de 2 min, README del repo
19:30–20:00	Verificar el entrypoint desde fuera y entregar